# SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-3-WS2025-CLCOM-EN/158471

# Semesterprojekt – DevOps und Cloud Computing 3B

Ausgeführt von: Benjamin Feichtlbauer, Kevin Forter,
Mikulovic Luka
Tepsurkaev Abdulah

Wien, den 8. Dezember 2025

# Inhaltsverzeichnis

# 1 Milestone 2

## 1.1 Network Setup

In this section, we describe the setup of the Virtual Private Cloud (VPC) and the associated network components within AWS.

### 1.1.1 VPC Configuration

We created a new VPC with the IPv4 CIDR block `10.0.0.0/16`. This provides a private network space for our infrastructure.

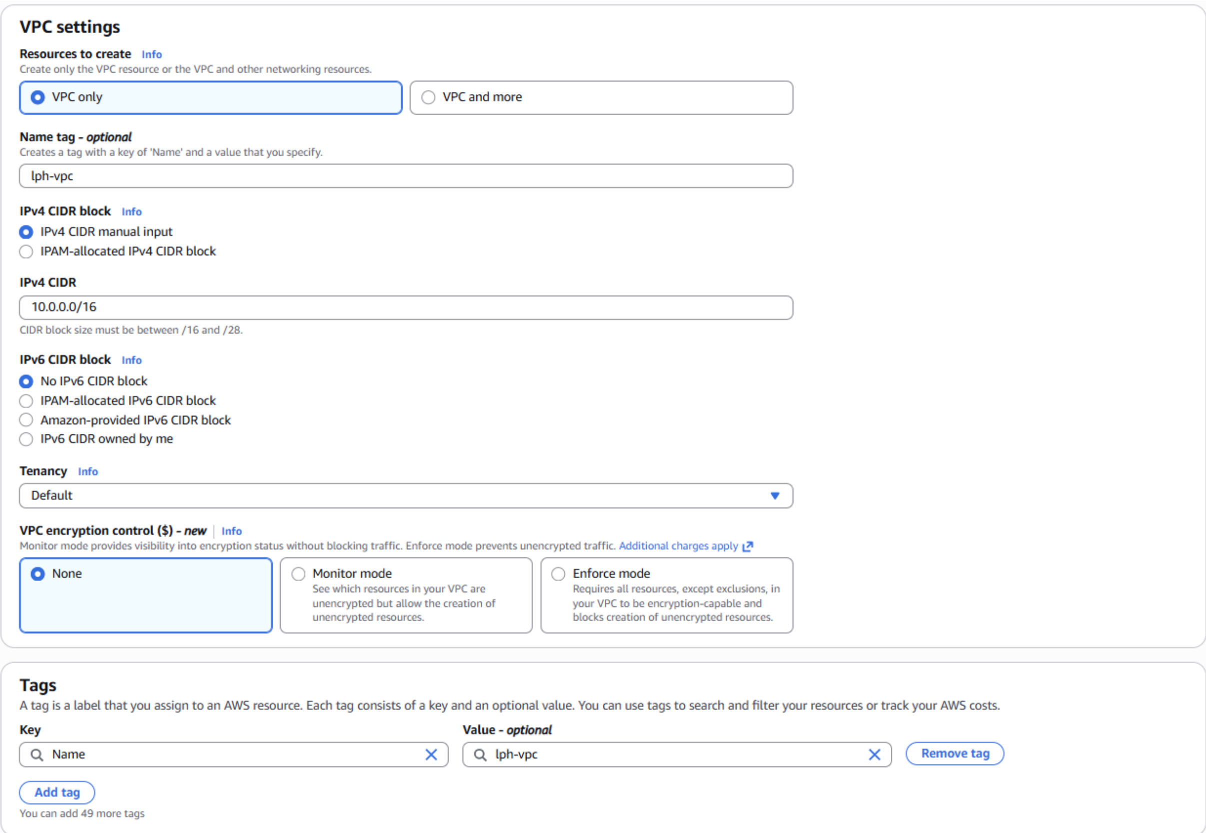



Abbildung 1: VPC Configuration

### 1.1.2 Subnets

A public subnet was created within the VPC with the CIDR block `10.0.0.0/24`. This subnet hosts our instances and allows them to communicate with each other and the internet.

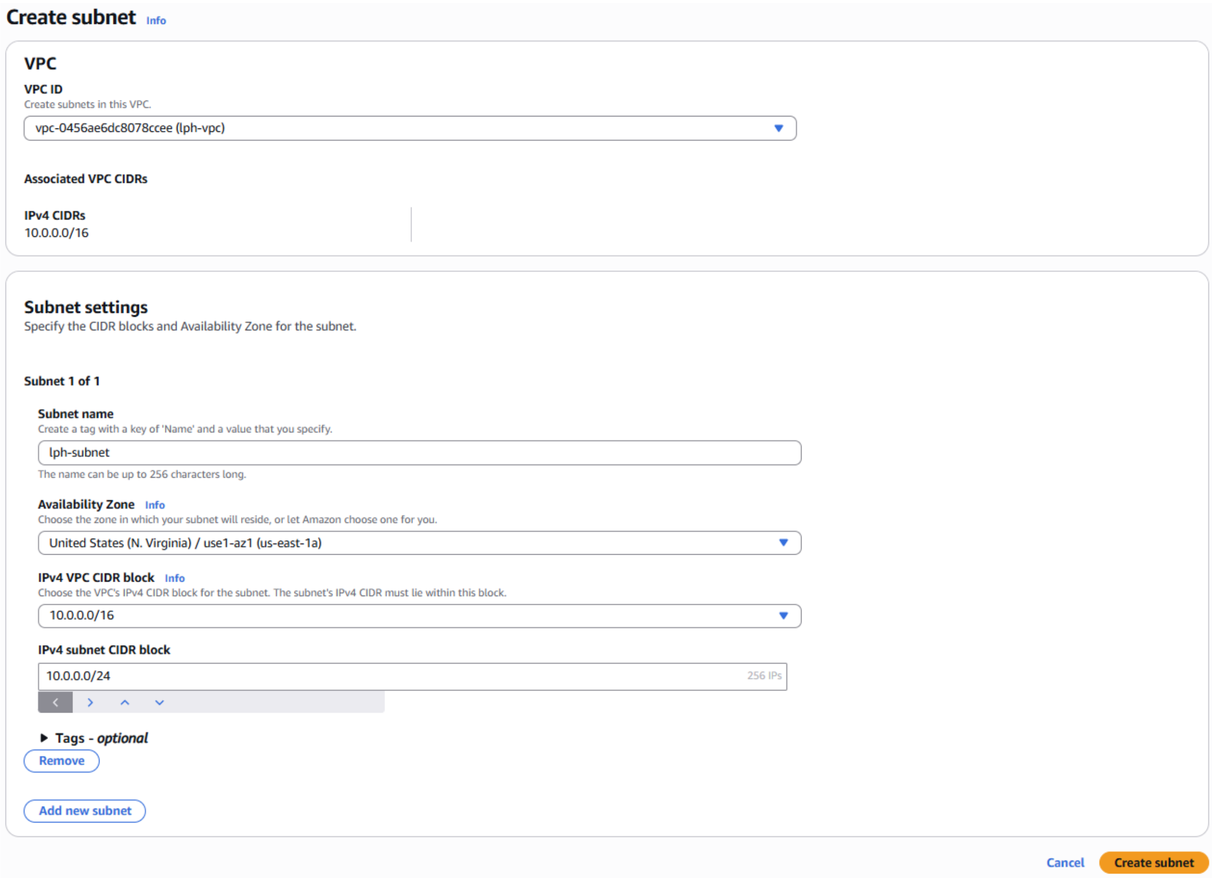



Abbildung 2: Subnet Configuration

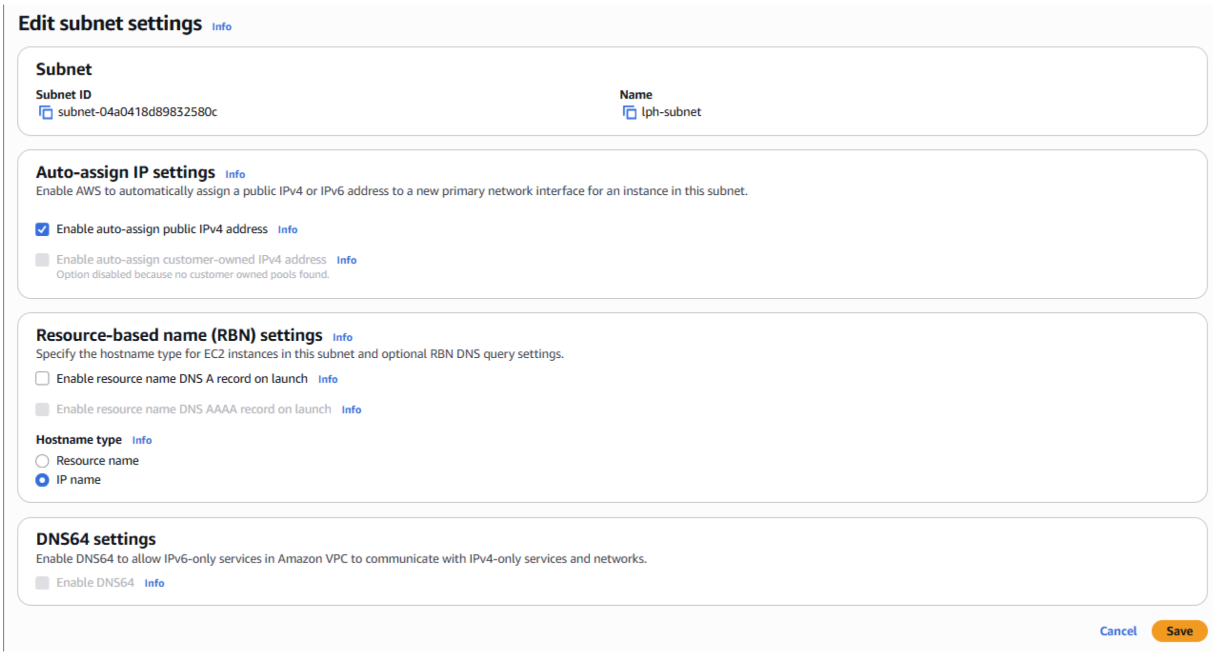



Abbildung 3: Subnet Settings

### 1.1.3 Internet Gateway

To enable internet access for the resources in our public subnet, we attached an Internet Gateway to the VPC.

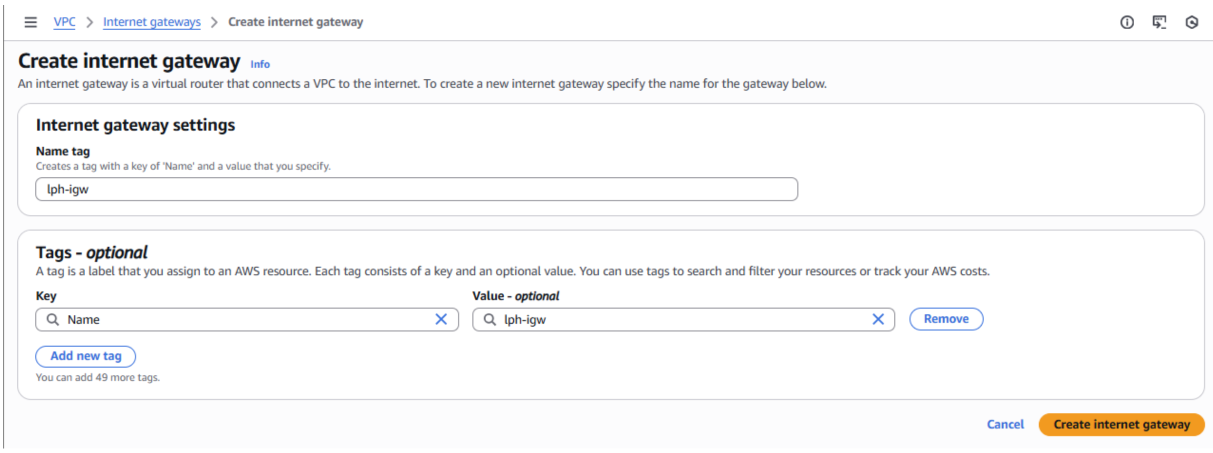



Abbildung 4: Internet Gateway

### 1.1.4 Route Tables

A custom route table was configured to direct traffic destined for the internet (`0.0.0.0/0`) to the Internet Gateway. This route table is associated with our public subnet.

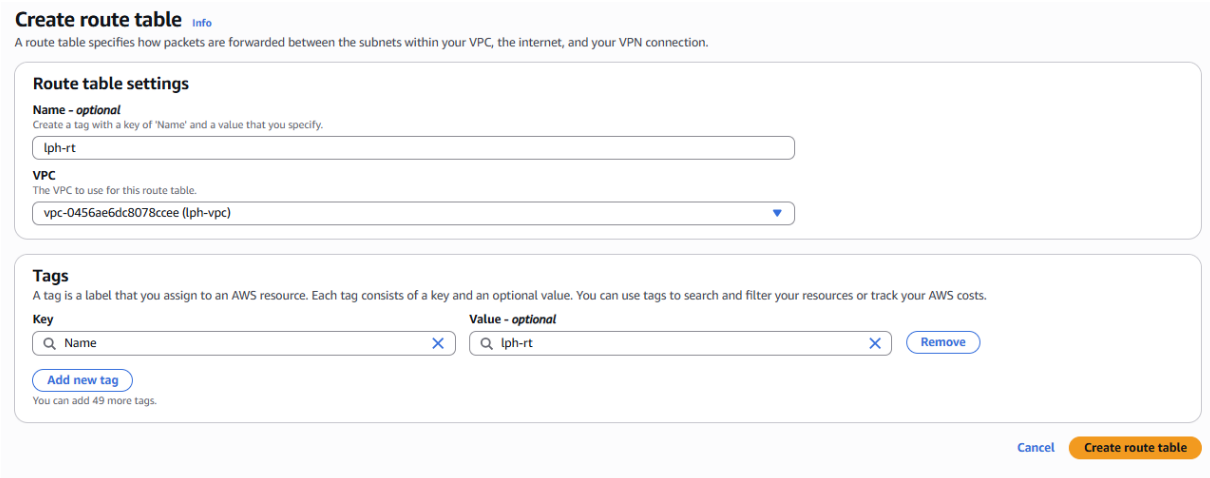



Abbildung 5: Route Table

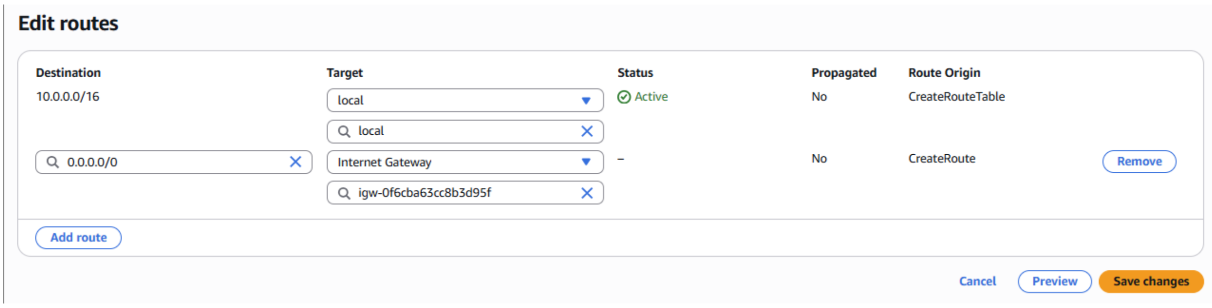



Abbildung 6: Route Table Default Route

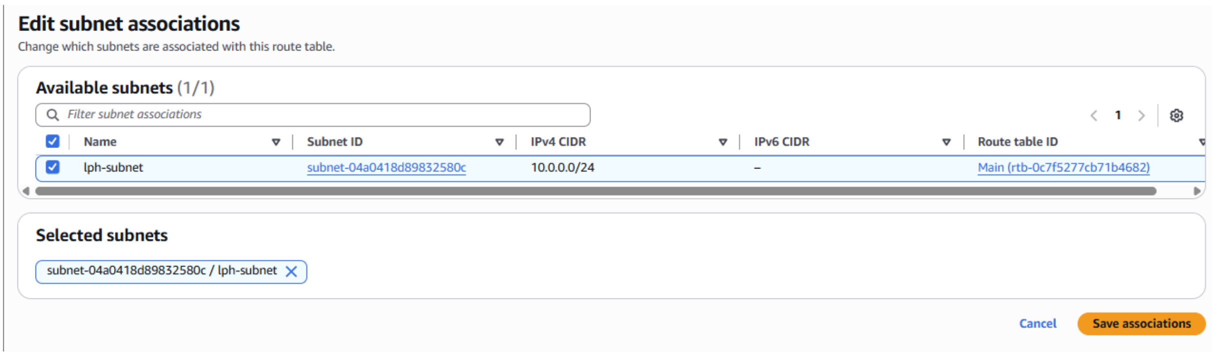



Abbildung 7: Route Table Subnet Association

### 1.1.5 Security Groups

Specific security groups were created for each component to control inbound and outbound traffic.

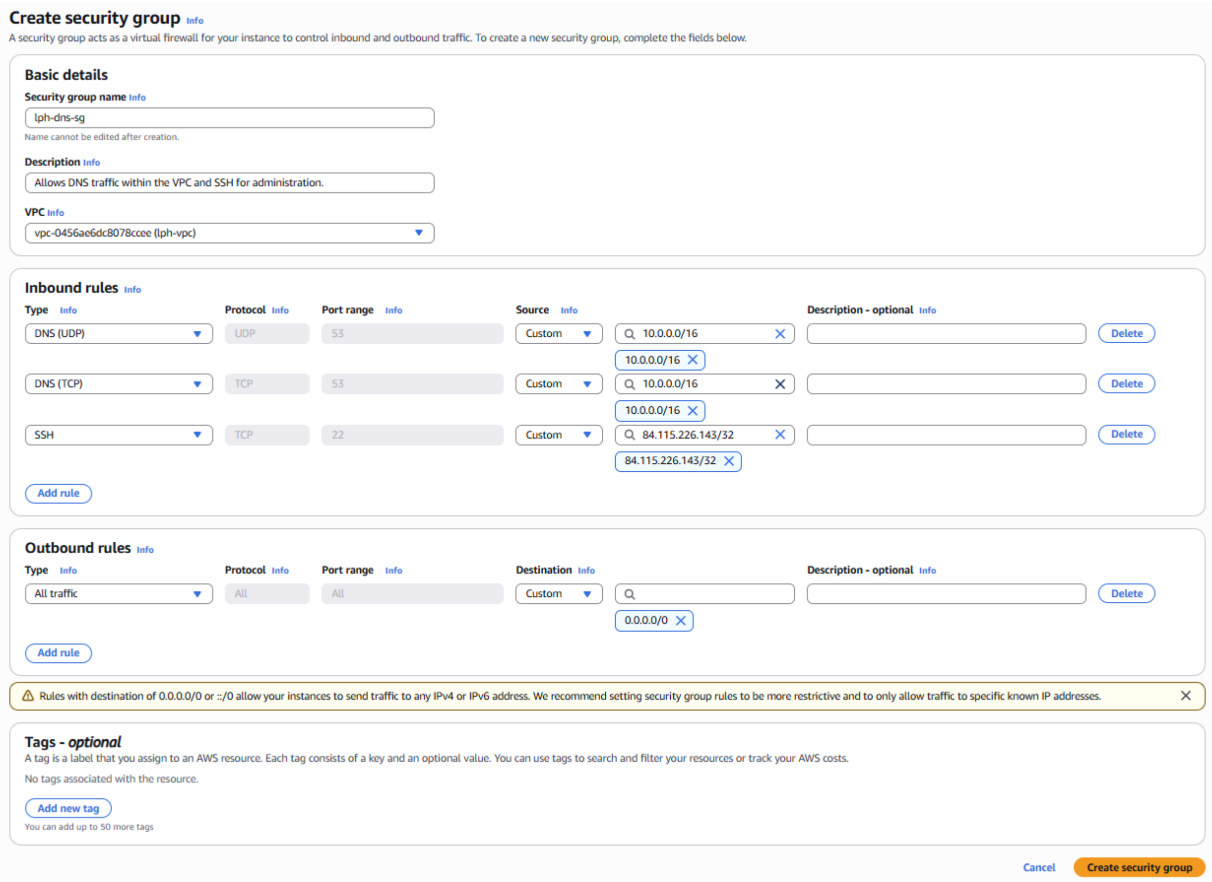



Abbildung 8: DNS Security Group

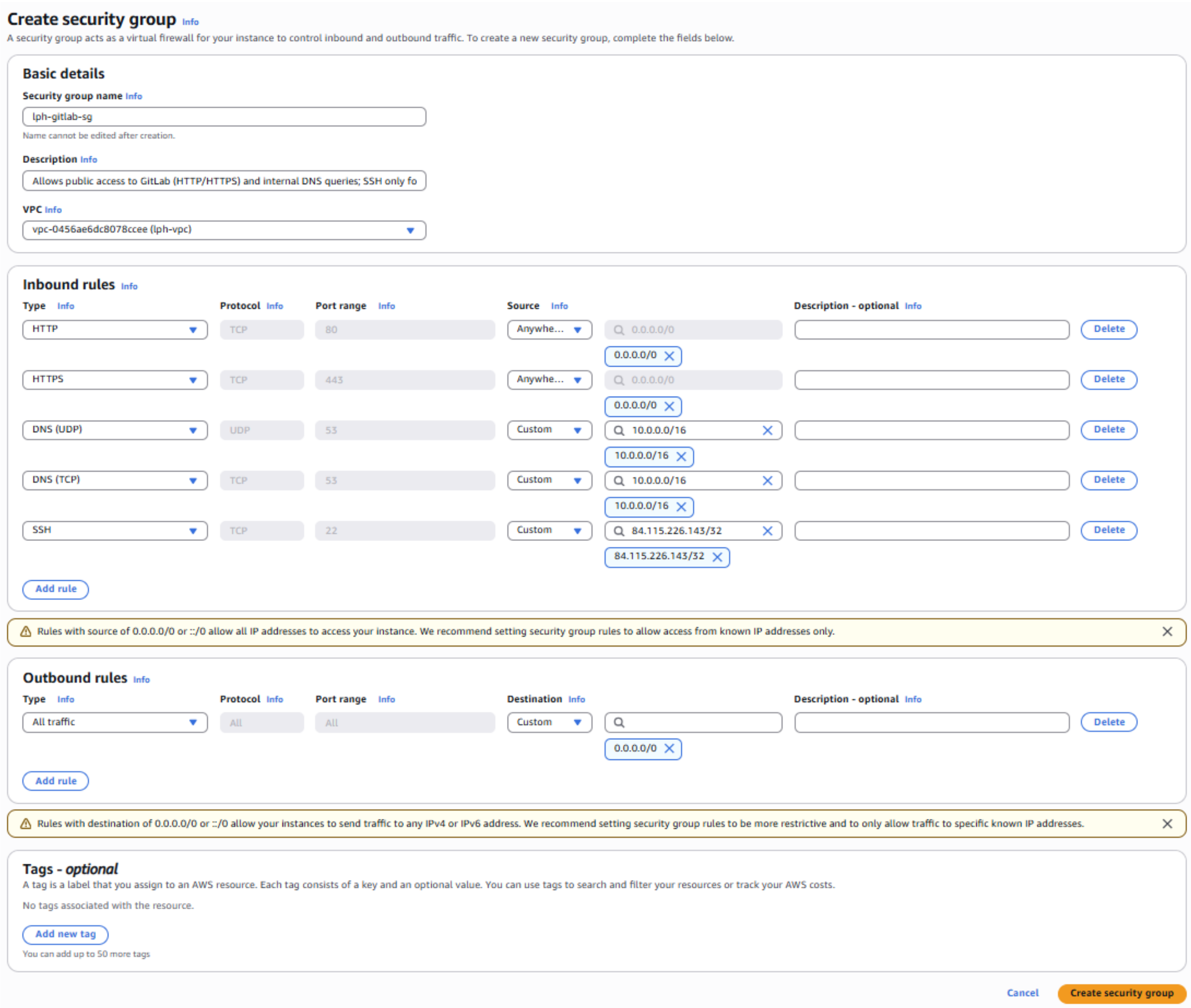



Abbildung 9: GitLab Security Group

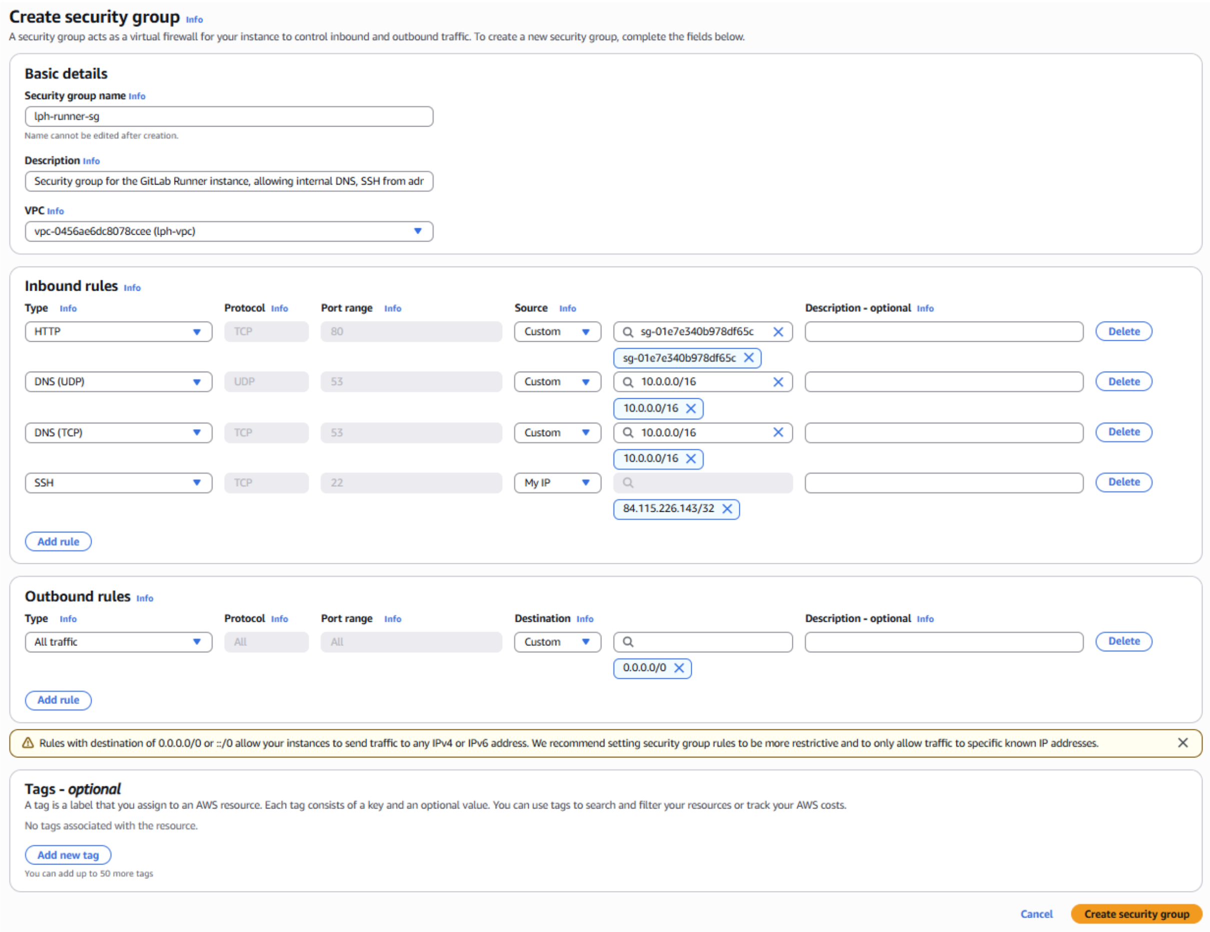



Abbildung 10: Runner Security Group

## 1.2 DNS Setup

We implemented a split-horizon DNS architecture using Bind9, with a primary and a secondary DNS server to ensure redundancy. The domain configured is `ci.intern`.

### 1.2.1 Primary DNS

The primary DNS server is hosted on an EC2 instance with the private IP `10.0.0.10`.

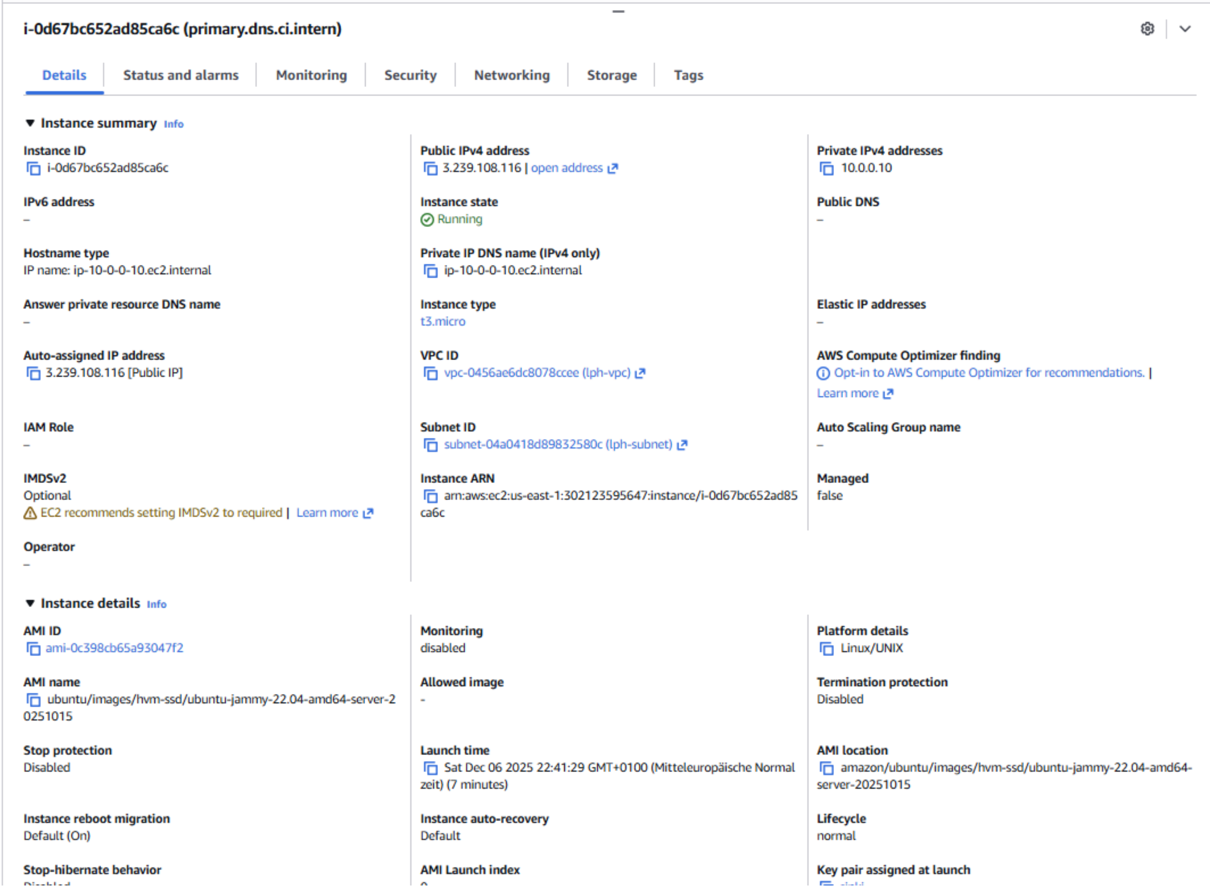



Abbildung 11: Primary DNS Instance Details

### Installation

We installed Bind9 on the Ubuntu 22.04 LTS instance:

```
sudo apt update
sudo apt install bind9 bind9utils -y
```

### Configuration

The zone configuration in `/etc/bind/named.conf.local` defines the forward and reverse lookup zones:

```
zone "ci.intern" {
    type master;
    file "/etc/bind/db.ci.intern";
    allow-transfer { 10.0.0.11; };
};

zone "0.0.10.in-addr.arpa" {
    type master;
```

```
    file "/etc/bind/db.10";
    allow-transfer { 10.0.0.11; };
};
```

The forward zone file `/etc/bind/db.ci.intern` maps hostnames to IP addresses:

```
$TTL 3600
@   IN  SOA primary.dns.ci.intern. admin.ci.intern. (
        2025010101 ; Serial
        3600       ; Refresh
        1800       ; Retry
        604800     ; Expire
        86400 )    ; Negative Cache TTL

    IN  NS  primary.dns.ci.intern.
    IN  NS  secondary.dns.ci.intern.

primary.dns     IN A 10.0.0.10
secondary.dns   IN A 10.0.0.11
gitlab          IN A 10.0.0.20
runner          IN A 10.0.0.21
```

The reverse zone file `/etc/bind/db.10` maps IP addresses back to hostnames:

```
$TTL 3600
@   IN  SOA primary.dns.ci.intern. admin.ci.intern. (
        2025010101
        3600
        1800
        604800
        86400 )

    IN NS primary.dns.ci.intern.
    IN NS secondary.dns.ci.intern.

10  IN PTR primary.dns.ci.intern.
11  IN PTR secondary.dns.ci.intern.
20  IN PTR gitlab.ci.intern.
21  IN PTR runner.ci.intern.
```

### 1.2.2 Secondary DNS

The secondary DNS server is hosted on an EC2 instance with the private IP `10.0.0.11`. It acts as a slave to the primary server.

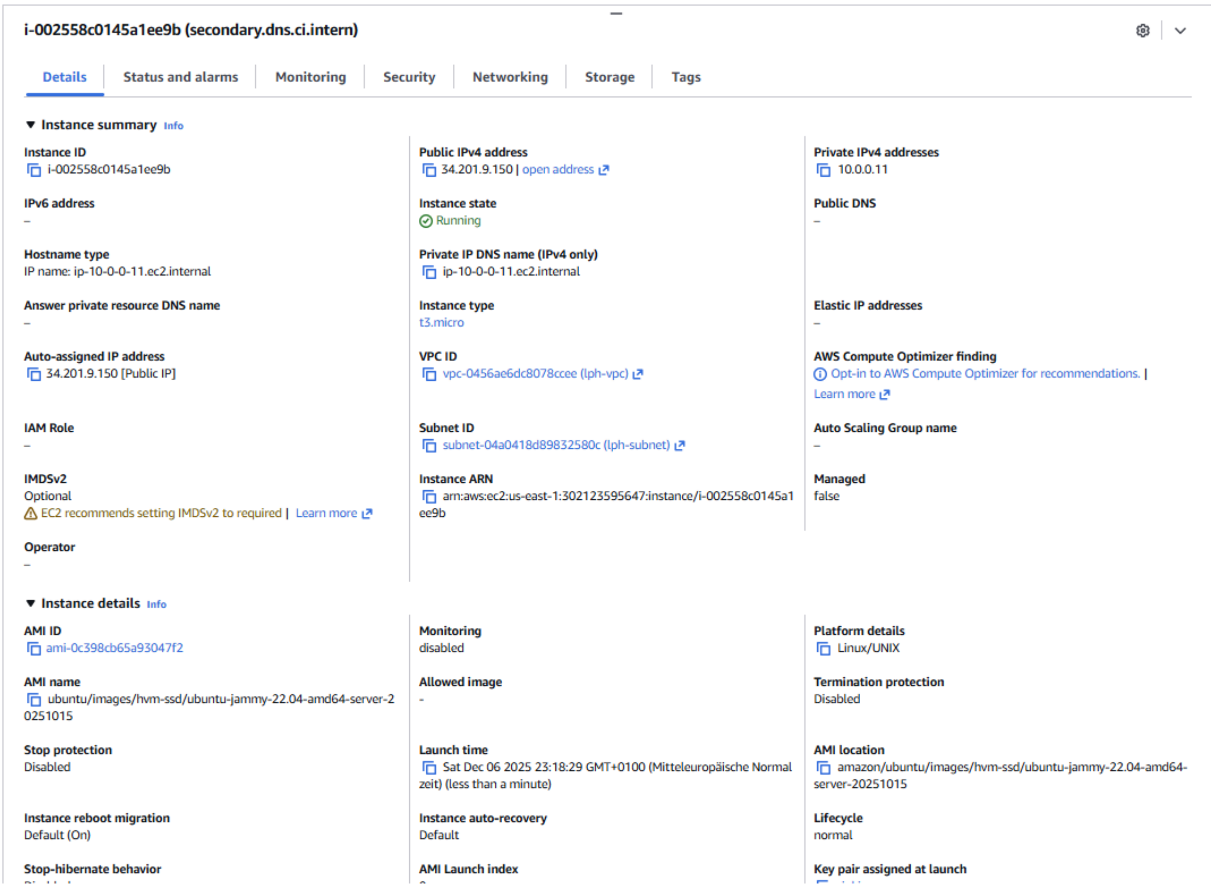



Abbildung 12: Secondary DNS Instance Details

### Configuration

The configuration in `/etc/bind/named.conf.local` specifies the master server for zone transfers:

```
zone "ci.intern" {
    type slave;
    masters { 10.0.0.10; };
    file "/var/cache/bind/db.ci.intern";
};

zone "0.0.10.in-addr.arpa" {
    type slave;
    masters { 10.0.0.10; };
    file "/var/cache/bind/db.10";
};
```

## 1.3 GitLab Server Setup

We deployed a self-hosted GitLab CE server using Docker on an EC2 instance with the private IP `10.0.0.20`.

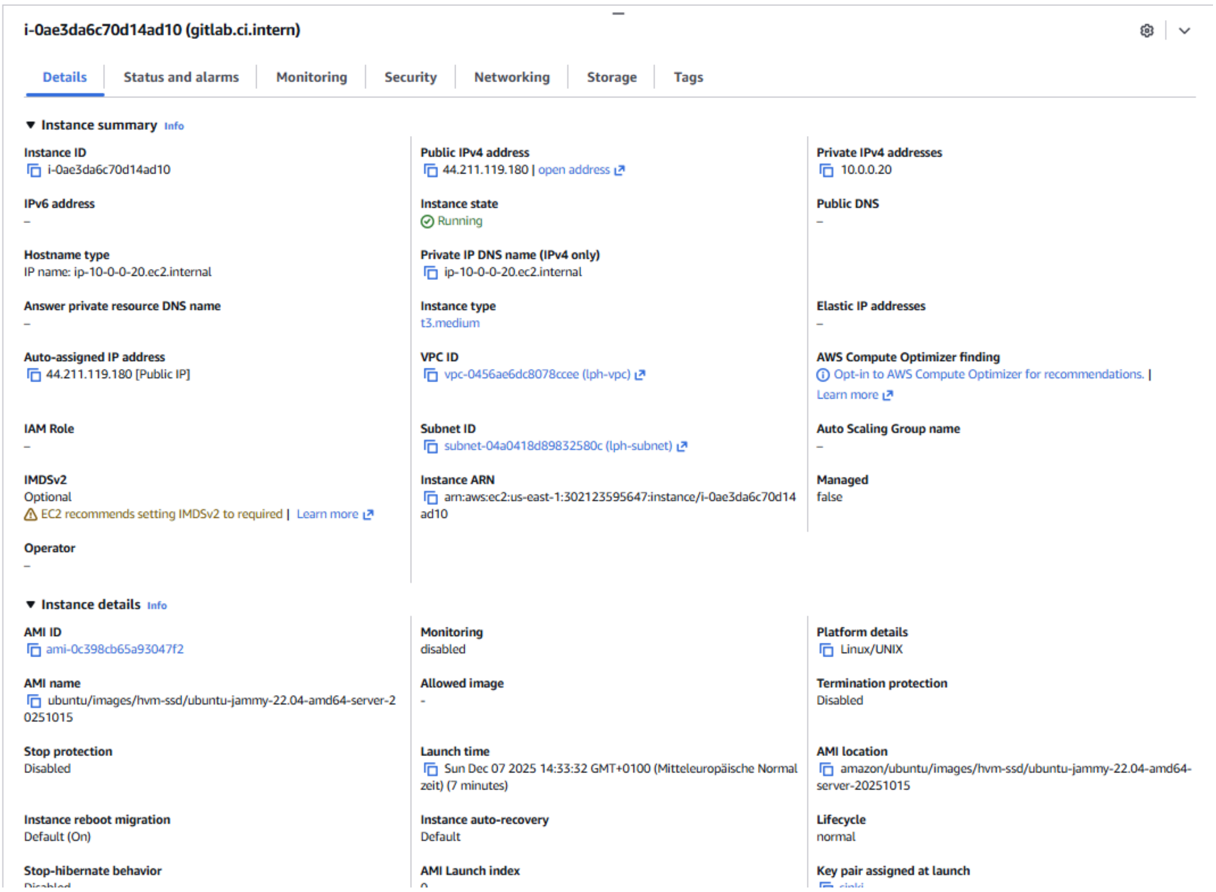



Abbildung 13: GitLab Server Instance Details

### 1.3.1 Installation

Docker and Docker Compose were installed, and a persistent volume structure was created. The `docker-compose.yml` configuration is as follows:

```
version: '3.7'
services:
  gitlab:
    image: gitlab/gitlab-ce:latest
    hostname: "gitlab.ci.intern"
    restart: always
    container_name: gitlab
    ports:
      - "80:80"
      - "443:443"
```

```
  volumes:
    - /srv/gitlab/config:/etc/gitlab
    - /srv/gitlab/logs:/var/log/gitlab
    - /srv/gitlab/data:/var/opt/gitlab
  environment:
    GITLAB_OMNIBUS_CONFIG: |
      external_url 'http://gitlab.ci.intern'
```

### 1.3.2 Verification

The GitLab instance is accessible via the browser using the public IP or the internal DNS name (within the network).

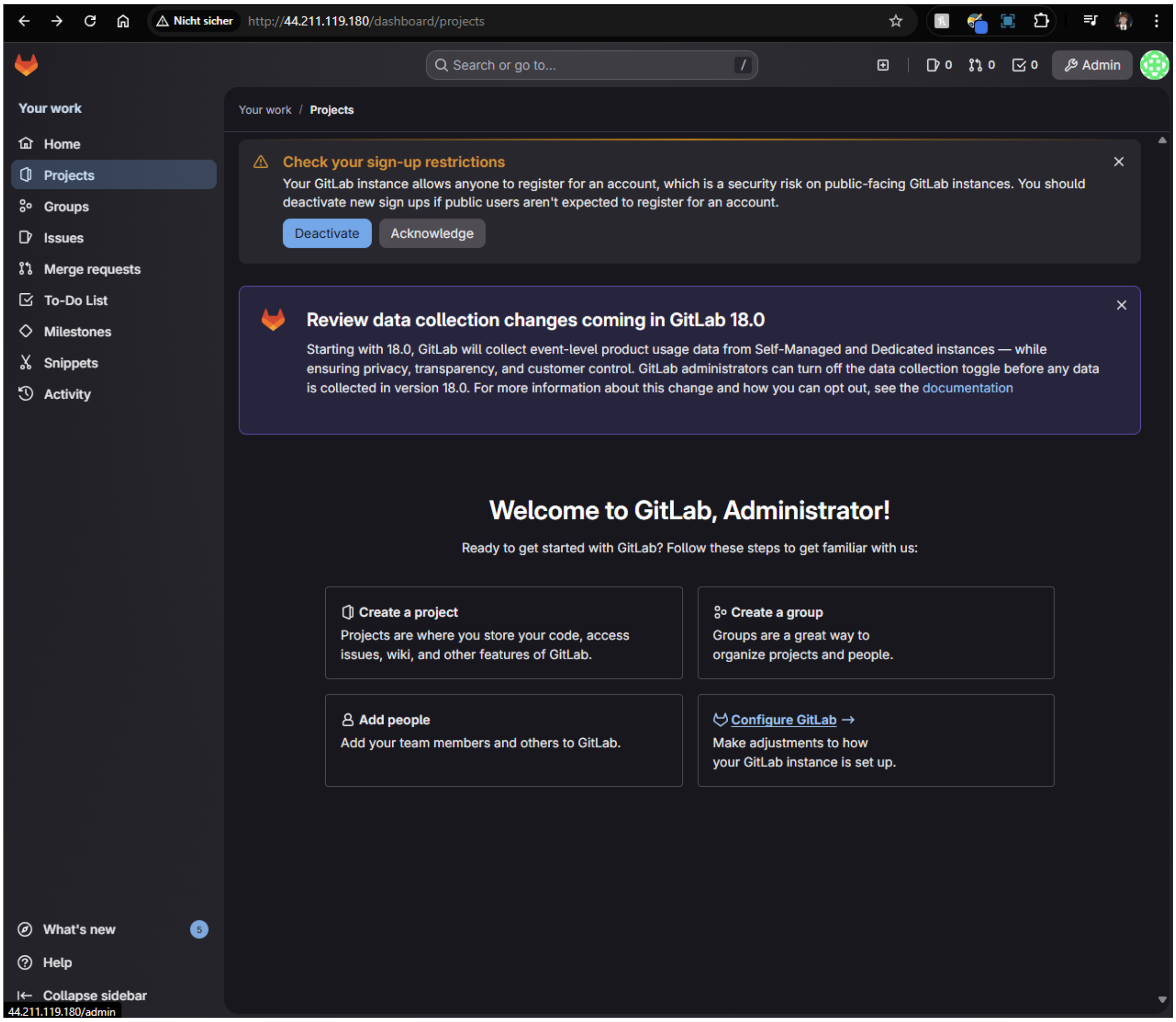



Abbildung 14: GitLab Homepage

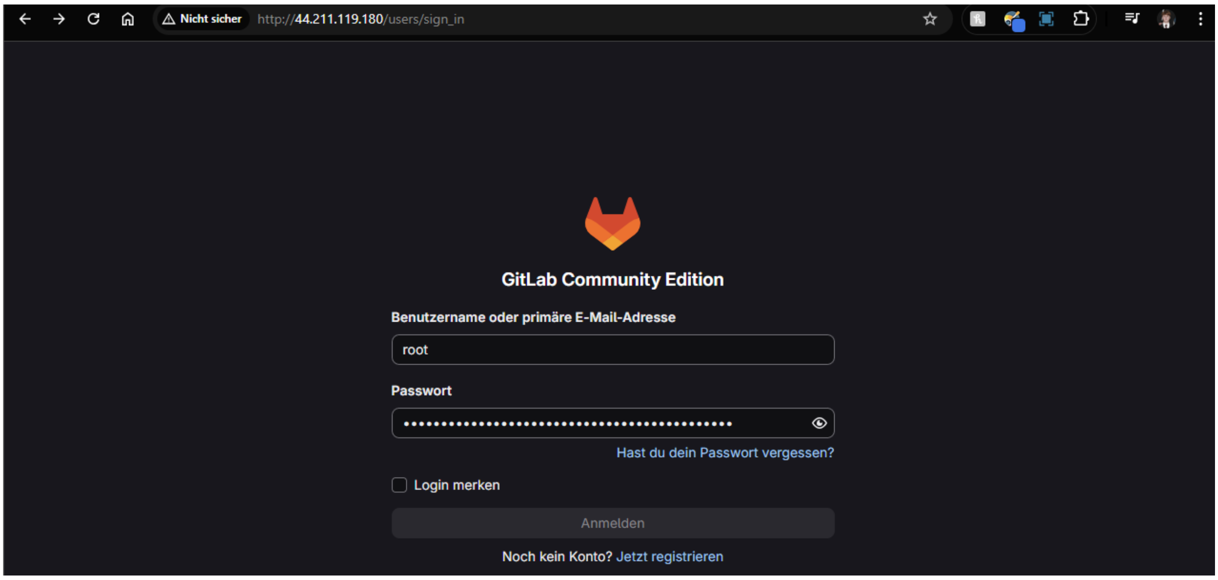



Abbildung 15: GitLab Login

### 1.3.3 System Configuration

The server runs on a `t3.medium` instance with 20 GB of gp3 storage to ensure sufficient resources for GitLab.

#### Swap Configuration

To prevent out-of-memory errors, we configured a 4GB swap file:

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

#### Initial Access

After installation, the initial root password was retrieved from the container configuration:

```
sudo cat /srv/gitlab/config/initial_root_password
```

## 1.4 GitLab Runner Setup

A GitLab Runner was set up on a separate EC2 instance (`10.0.0.21`) to execute CI/CD jobs.

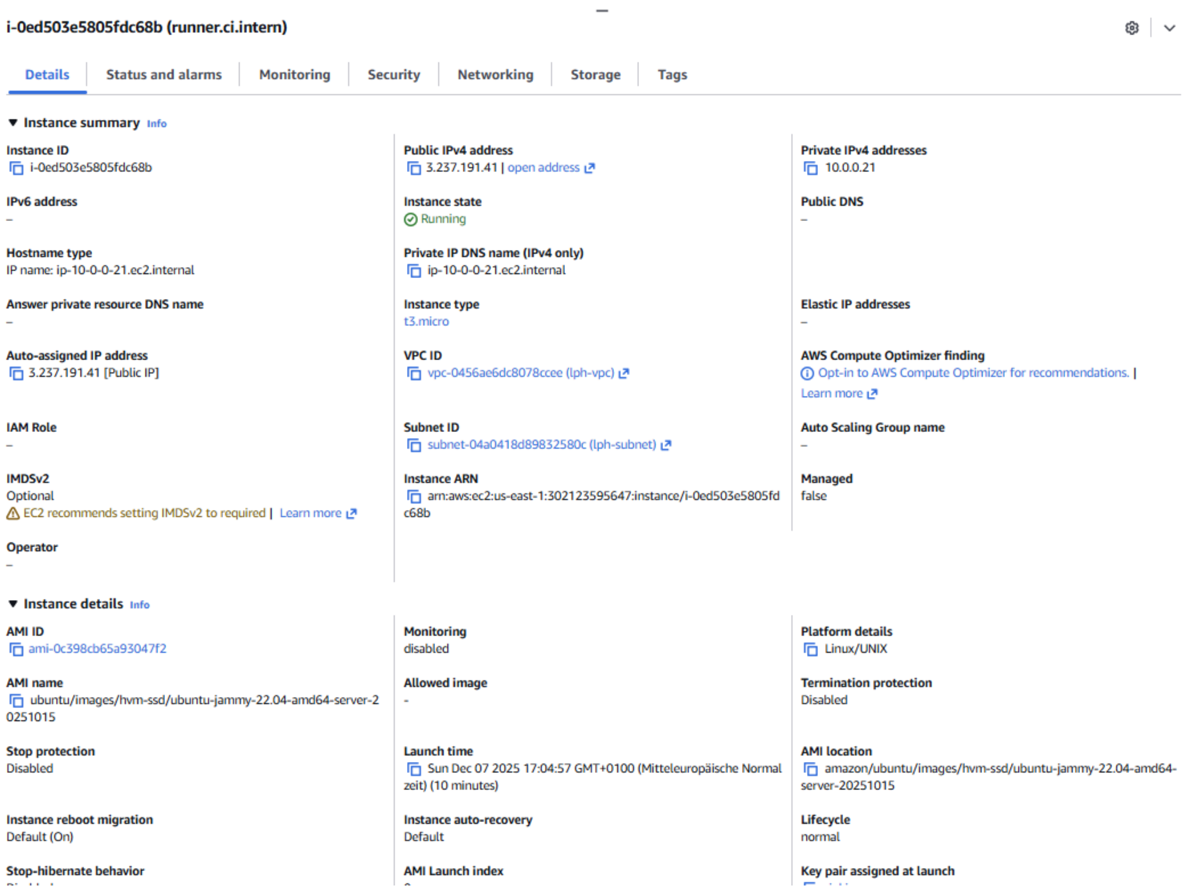



Abbildung 16: GitLab Runner Instance Details

### 1.4.1 Configuration

The runner was configured to use the internal DNS servers to resolve the GitLab server's hostname.

```
sudo systemctl disable --now systemd-resolved
sudo rm /etc/resolv.conf
echo "nameserver 10.0.0.10" | sudo tee /etc/resolv.conf
echo "nameserver 10.0.0.11" | sudo tee -a /etc/resolv.conf
```

The runner container was started with specific DNS settings:

```
sudo docker run -d --name gitlab-runner \
  --restart always \
  -v /srv/gitlab-runner/config:/etc/gitlab-runner \
  --dns 10.0.0.10 \
  --dns 10.0.0.11 \
  gitlab/gitlab-runner:latest
```

### 1.4.2 Registration

The runner was registered with the GitLab server using the internal URL `http://gitlab.ci.intern`.

```
sudo docker exec -it gitlab-runner gitlab-runner register
# URL: http://gitlab.ci.intern
# Executor: docker
# Default Docker image: alpine:latest
```

## 1.5 GitLab Workflow Verification

To ensure the GitLab server and Runner are functioning correctly, we performed a complete workflow test involving repository creation, code changes, and pipeline execution.

### 1.5.1 Repository Creation

We created a new blank project named `test-runner` in the GitLab web interface.

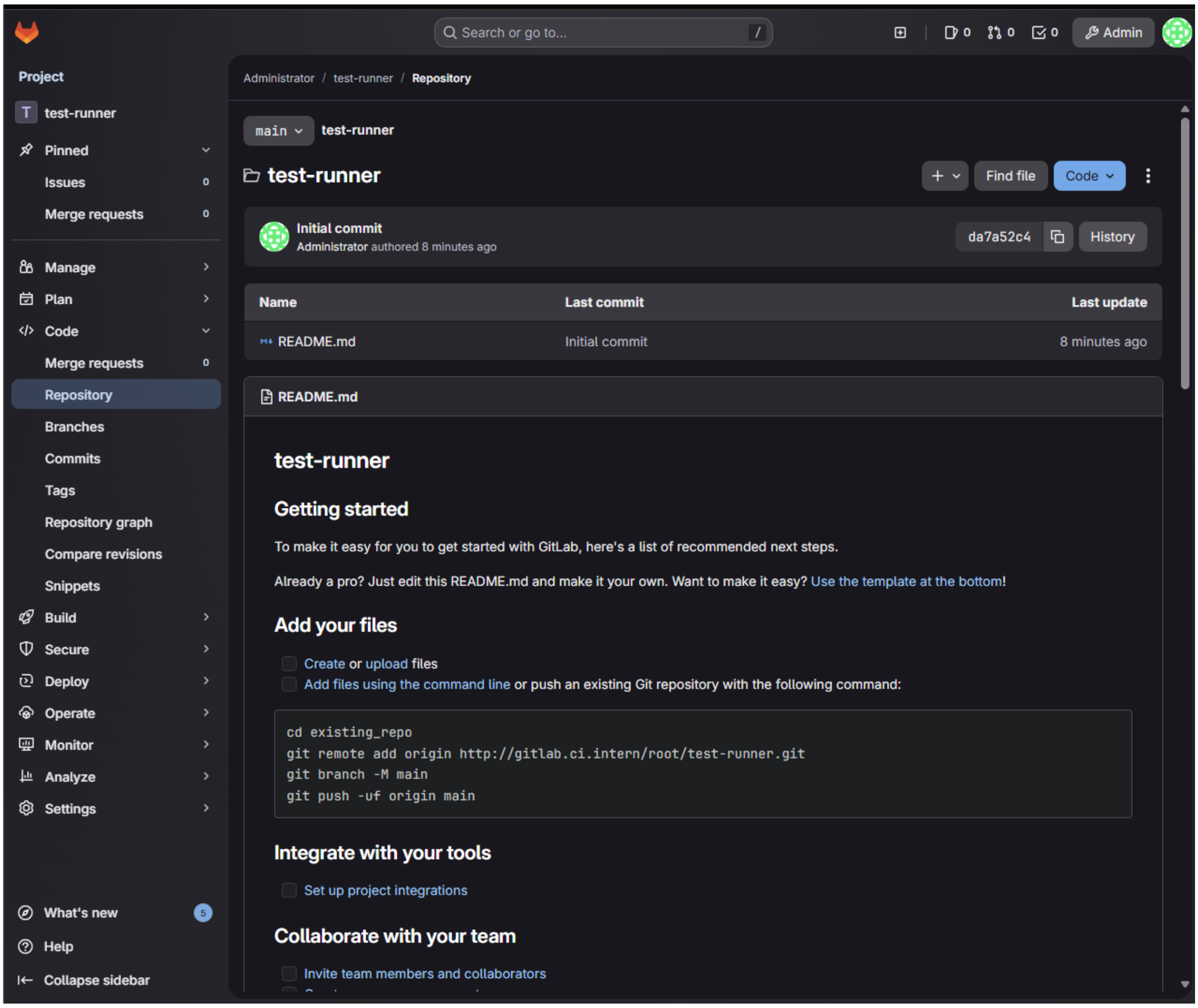



Abbildung 17: Creating a New Project

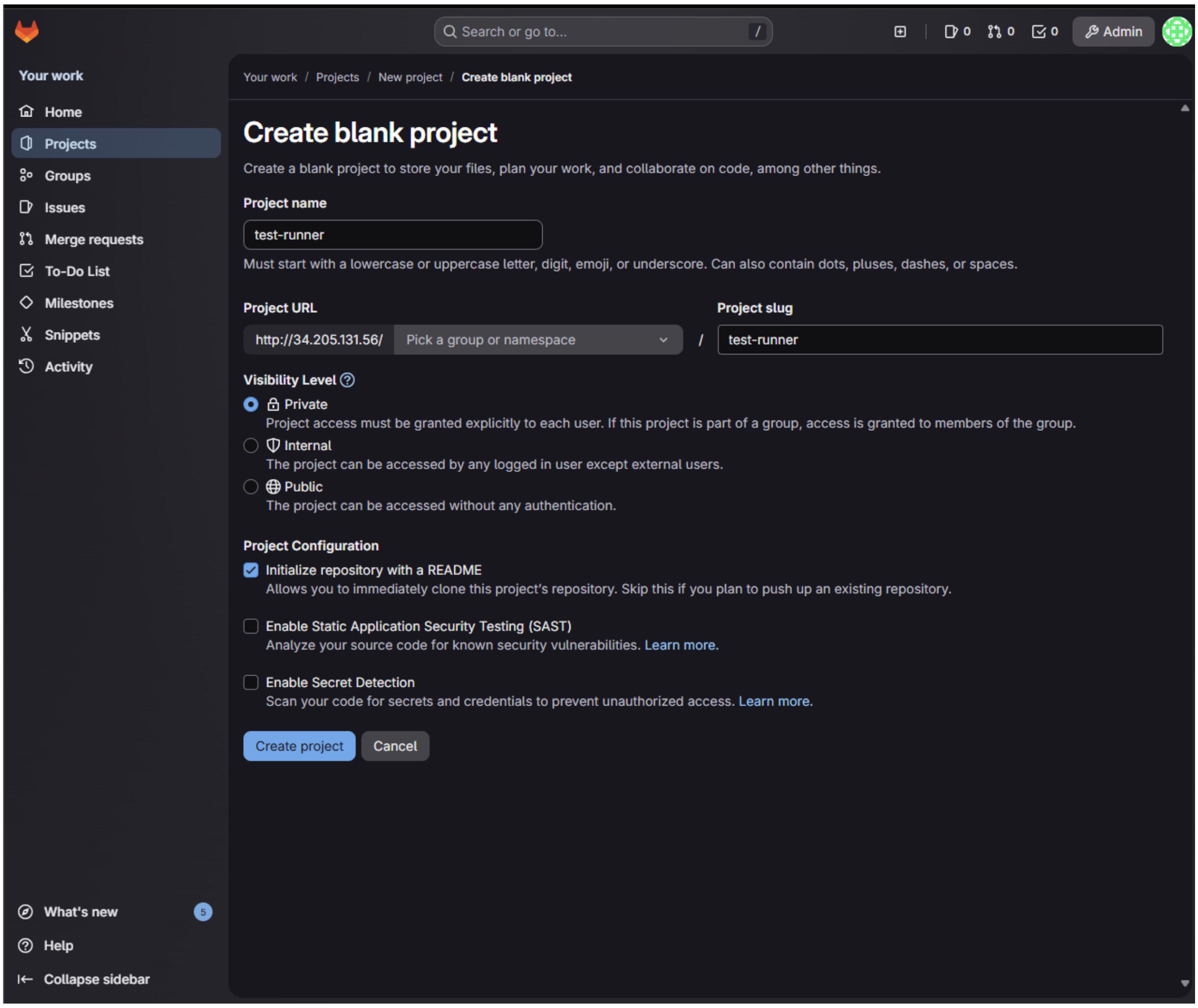



Abbildung 18: Initialized Empty Repository

### 1.5.2 SSH Configuration

To facilitate secure communication between the local machine and the GitLab server, we generated an SSH key pair and added the public key to the GitLab user profile.

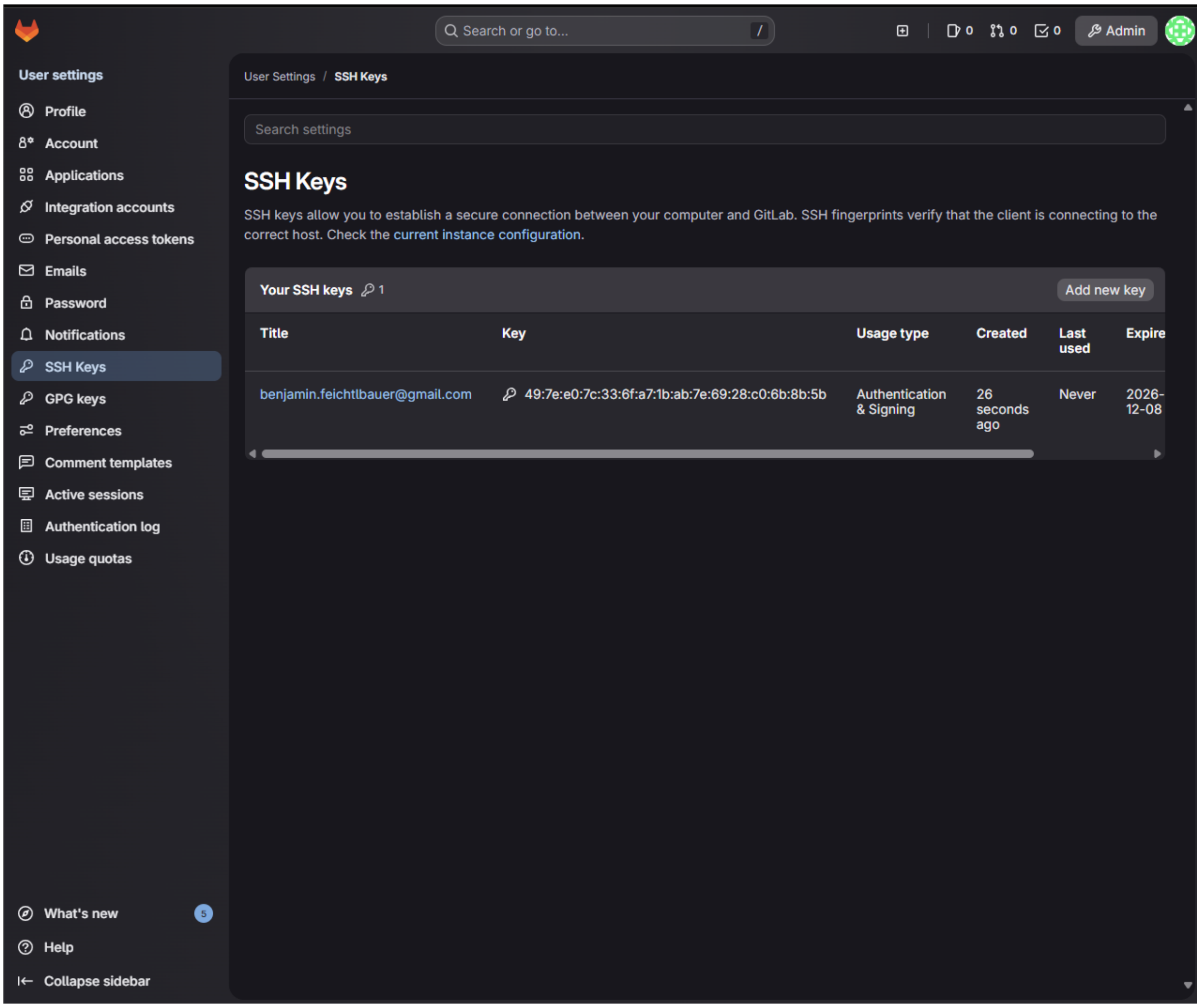



Abbildung 19: Adding SSH Key

### 1.5.3 Cloning and Committing

Since the local machine is outside the AWS VPC, we cloned the repository using the GitLab server's public IP address.

```
git clone http://<PUBLIC_IP>/root/test-runner.git
cd test-runner
```

We then created a test file `demo.txt` and committed it to the repository.

```
echo "test run $(date)" >> demo.txt
git add .
git commit -m "demo pipeline test"
```

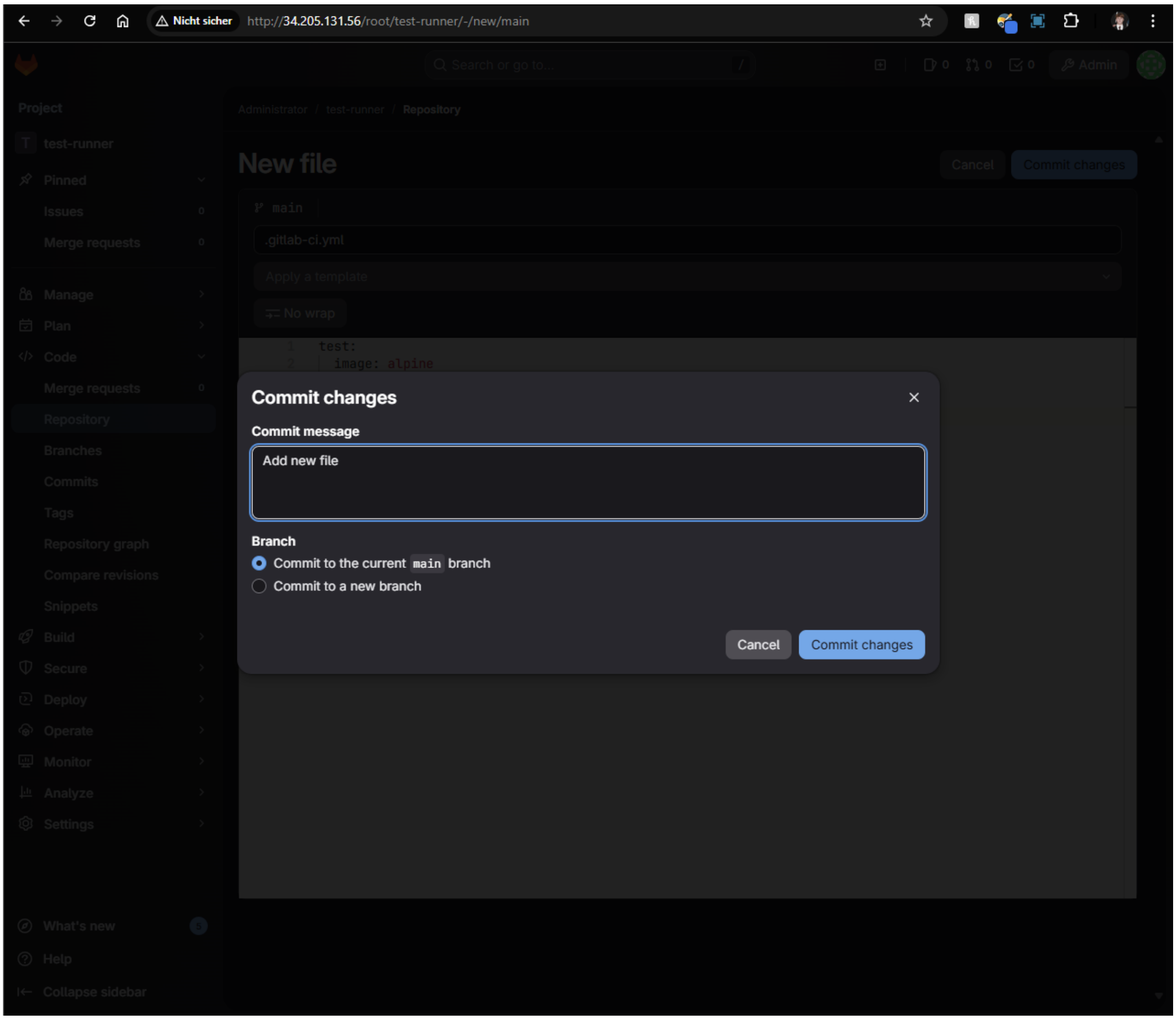
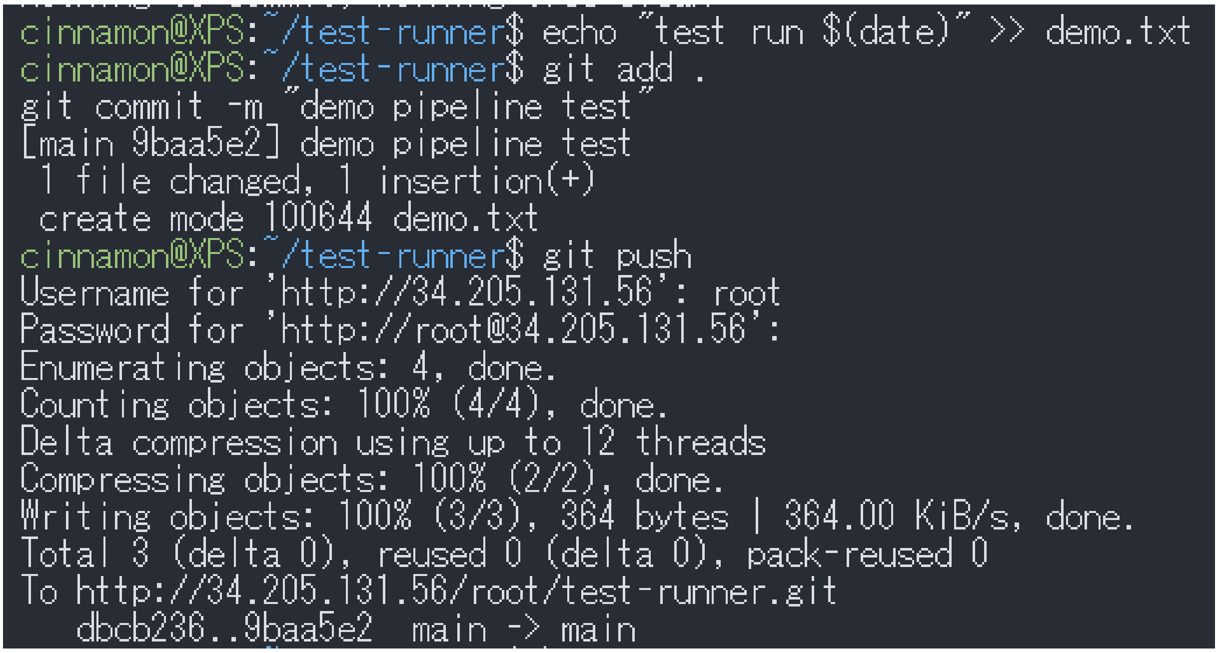



Abbildung 20: Git Commit

### 1.5.4 Pushing Changes

The changes were pushed to the remote repository.

```
git push
```



```
cinnamon@XPS:~/test-runner$ echo "test run $(date)" >> demo.txt
cinnamon@XPS:~/test-runner$ git add .
git commit -m "demo pipeline test"
[main 9baa5e2] demo pipeline test
 1 file changed, 1 insertion(+)
 create mode 100644 demo.txt
cinnamon@XPS:~/test-runner$ git push
Username for 'http://34.205.131.56': root
Password for 'http://root@34.205.131.56':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 364 bytes | 364.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To http://34.205.131.56/root/test-runner.git
   dbcb236..9baa5e2  main -> main
```

Abbildung 21: Git Push

After the push, the new file is visible in the repository, and the CI/CD pipeline is automatically triggered.

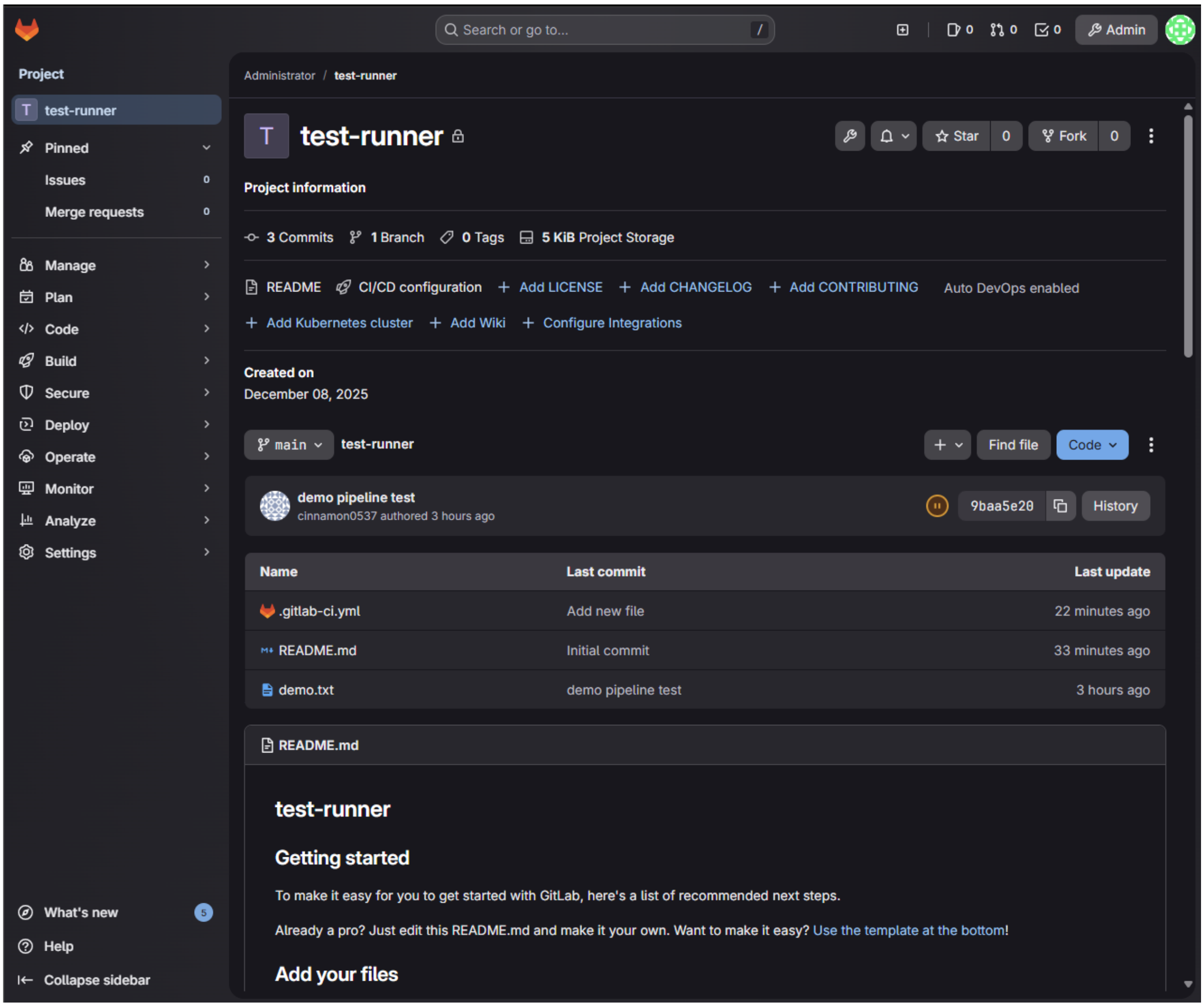



Abbildung 22: Repository after Push